



HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

The background of the slide is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint, stylized 'S' shape that curves around the central text. The dots are of varying sizes and are more densely packed in some areas than others, creating a textured, digital effect.

SOICT

School of Information and Communication Technology

ONE LOVE. ONE FUTURE.



TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Natural Language Processing

3 – Transformer and Pre-trained Language Models

ONE LOVE. ONE FUTURE.

FYI

- *Lecturer: Dr. Bui Thi Mai Anh*
- **Research domains:**
 - **Software Engineering**
 - **Applied AI in Software Engineering**
 - **Search-based Software Engineering:**
evolutionary computing,
reinforcement learning, deep learning
 - **NLP4SE**
- **Lab: Intelligent Software Engineering – BK-AI Center**

<https://iselab-dorlab.github.io/>



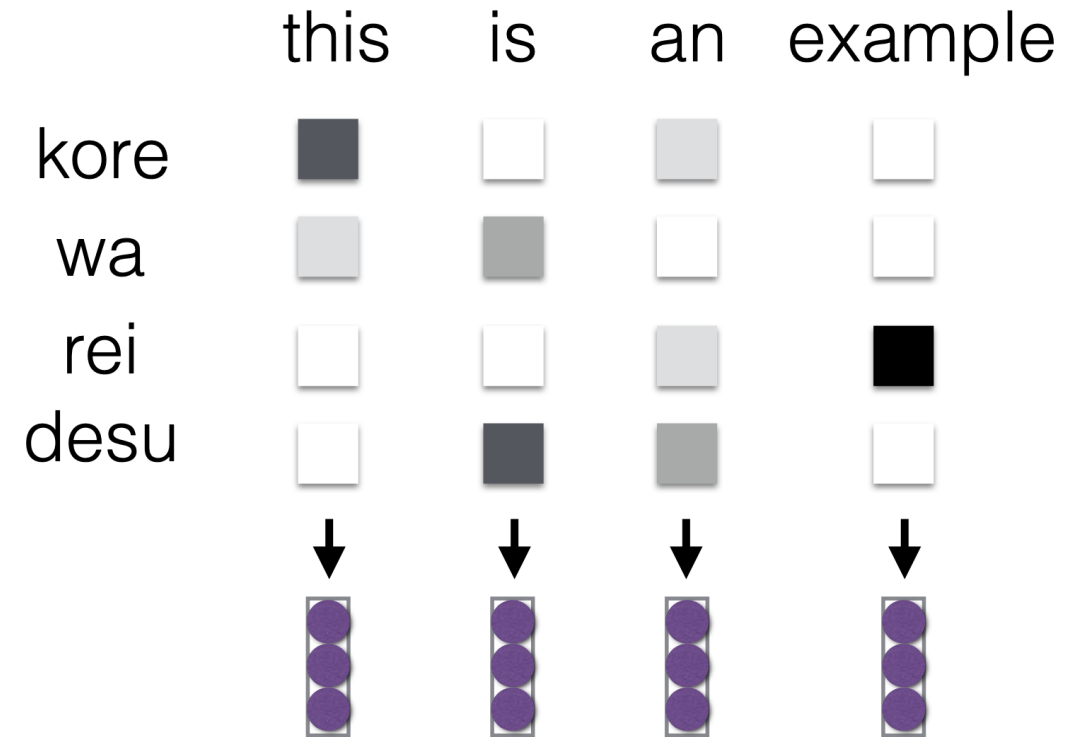


Section 1 - Transformer

ONE LOVE. ONE FUTURE.

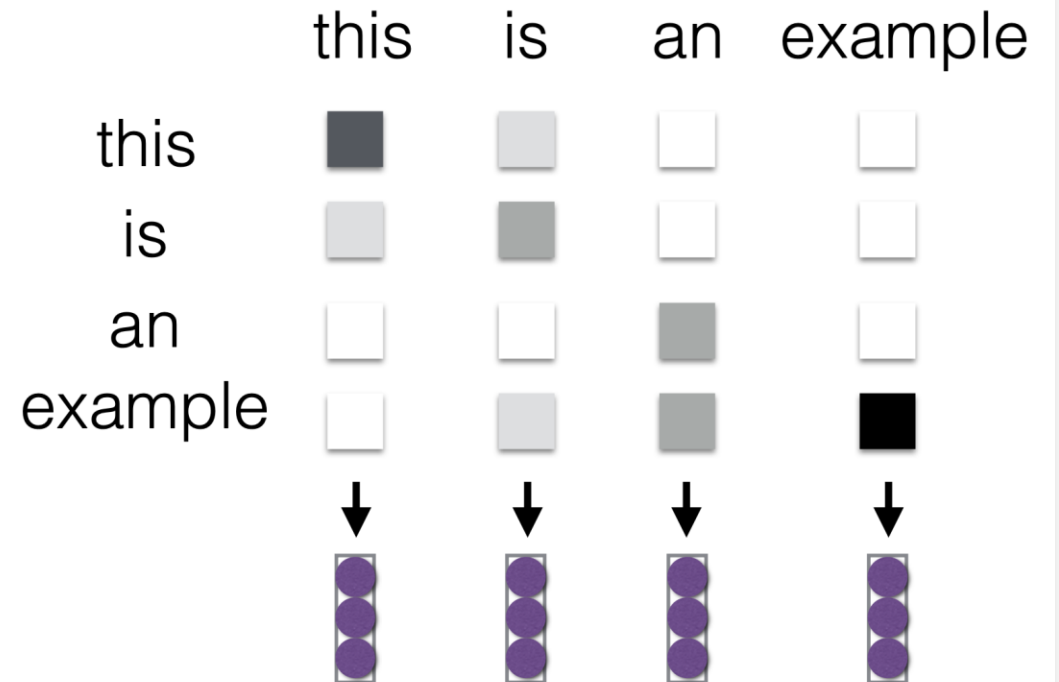
Cross Attention

- Each element in a sequence attends to elements of another sequence



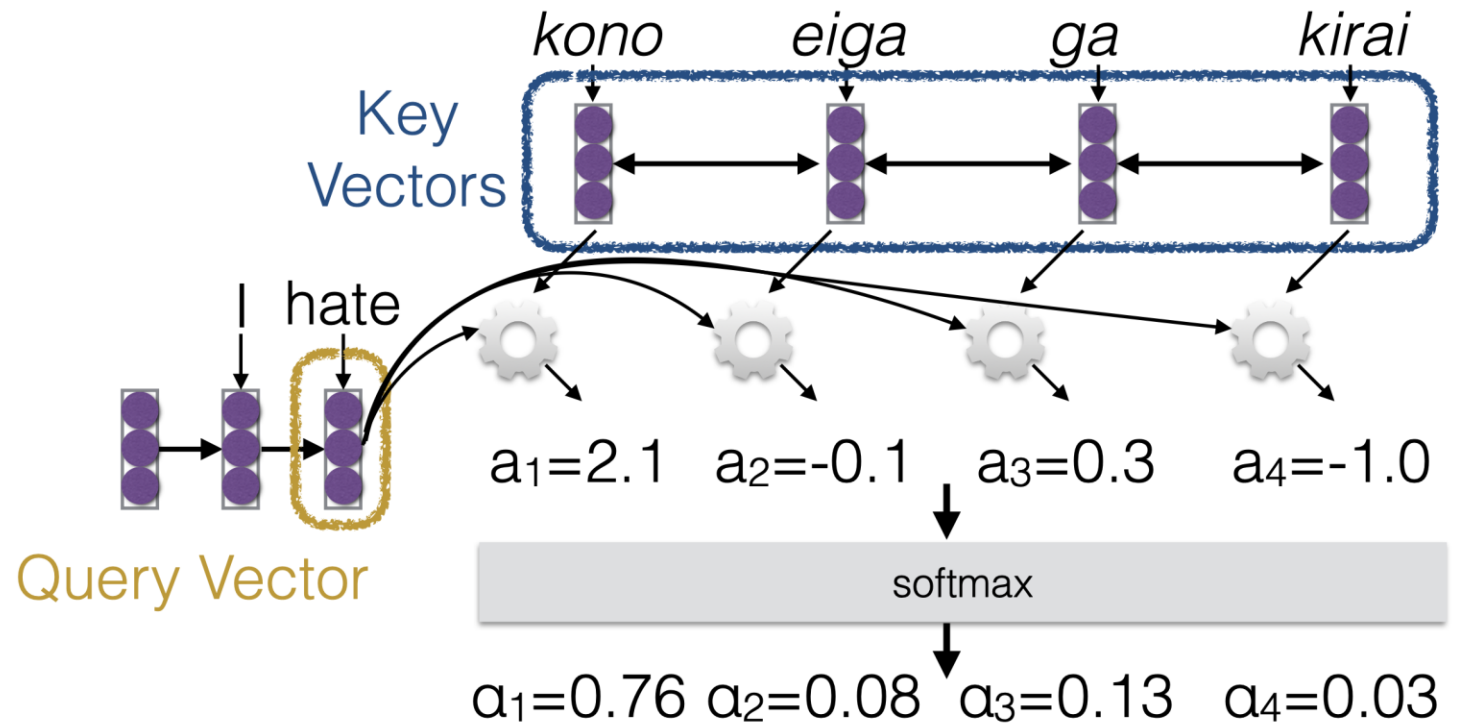
Self Attention

- Each element in the sequence attends to elements of that sequence
→ context sensitive encodings



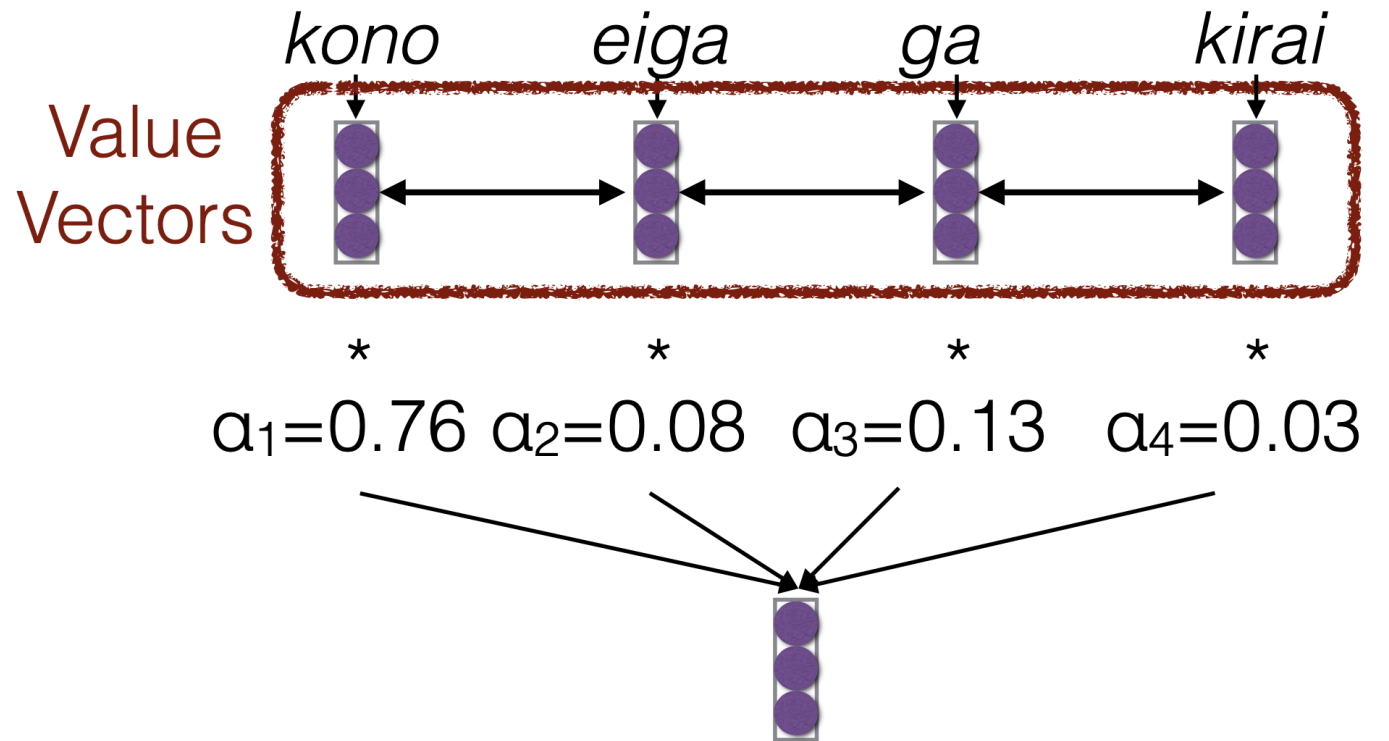
Calculate Attention (1)

- Use **query** vector (decoder state) and **key** vectors (all encoder states)
- For each **query-key** pair, calculate **weight**
- Normalize to add to one using soft-max



Calculate Attention (2)

- Combine together **value vectors** (usually encoder states, like key vectors) by taking the **weighted sum**
- Use this in any part of the model you like



Transformer – Attention is All You Need

- A sequence to sequence model based entirely on Attention
- Strong results on machine translation
- Fast: only matrix multiplication

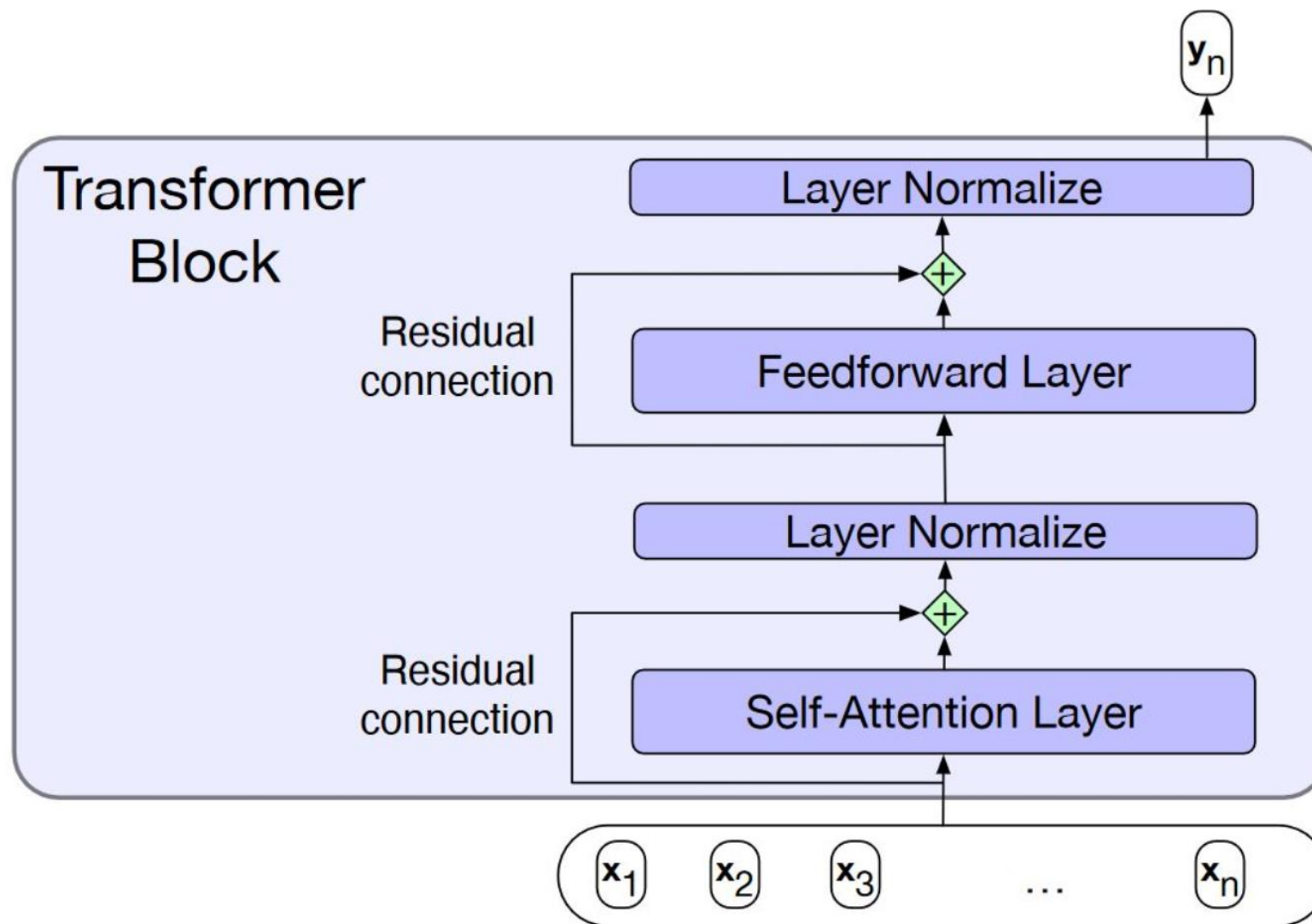
Transformer – Self Attention

- Query: the **current focus of attention** when being compared to all other preceding inputs
- Key: a **preceding input** being compared to the current focus of attention
- Value: the **output** of the **current focus of attention**
- Weight matrices of Transformer: W^Q, W^K, W^V

$$\mathbf{q}_i = W^Q \mathbf{x}_i; \quad \mathbf{k}_i = W^K \mathbf{x}_i; \quad \mathbf{v}_i = W^V \mathbf{x}_i$$

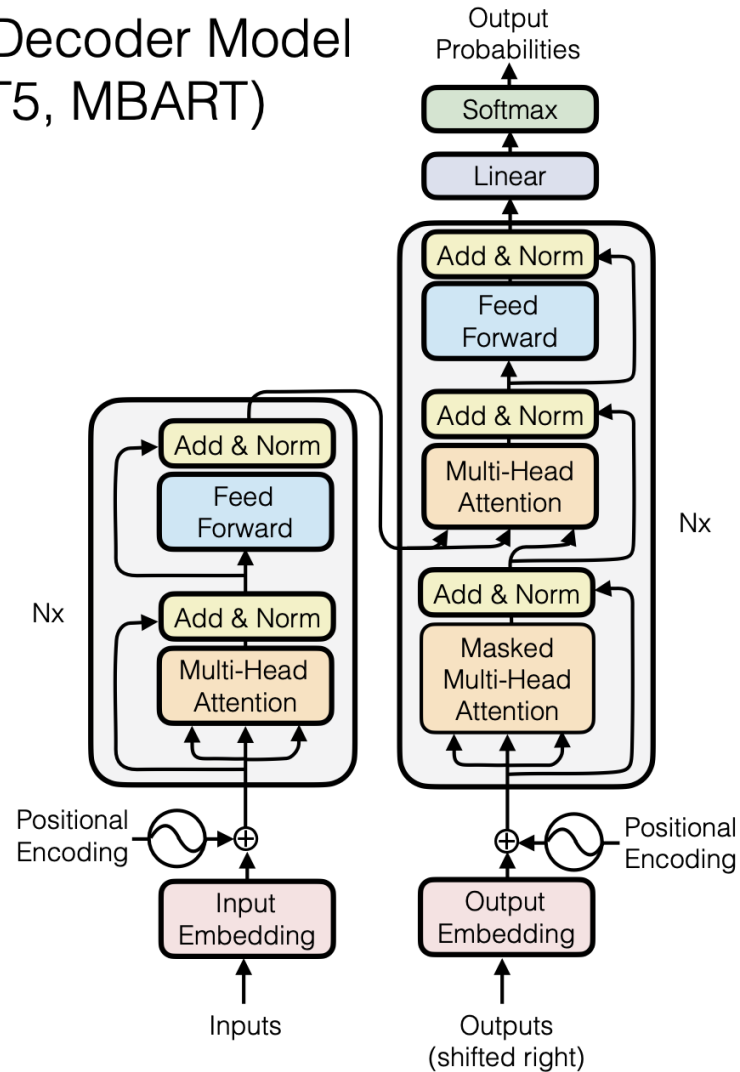
- The input x , the output y of Transformer, have the same dimensionality $1 \times d$

A Transformer Block

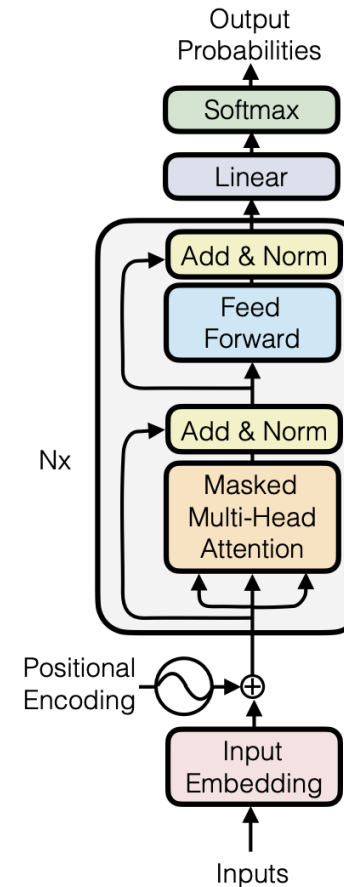


Two types of Transformer

Encoder-Decoder Model
(e.g. T5, MBART)



Decoder Only Model
(e.g. GPT, LLaMa)



Core Transformer concepts

- Input Embedding / Output Embedding
- Positional Encodings
- Multi-head attention
- Masked attention
- Residual + Layer normalization
- Feed-forward layer

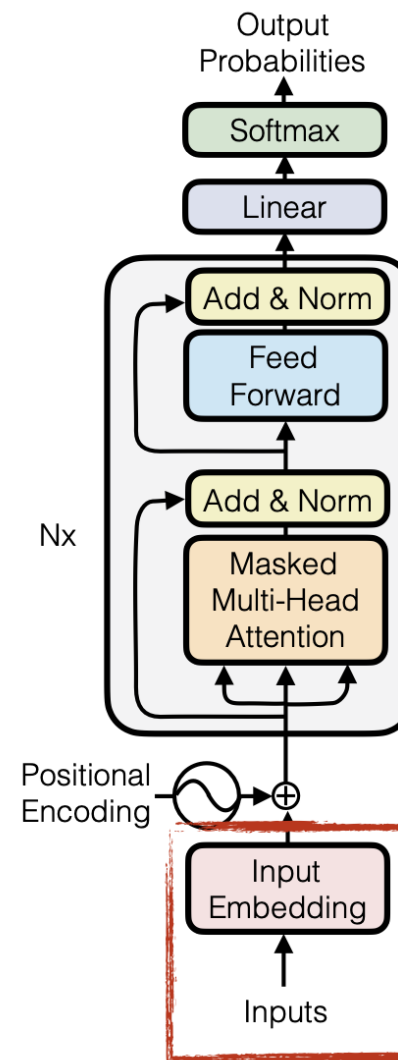
Input/Output Embeddings

- **Inputs:** Generally split using subwords

the books were improved

the book _s were **improv _ed**

- **Input Embedding:** Looked up, like in previously discussed models



Multi-head Attention

- **Intuition:** Information from different parts of the sentence can be useful to disambiguate in different ways

I **run** a small **business**

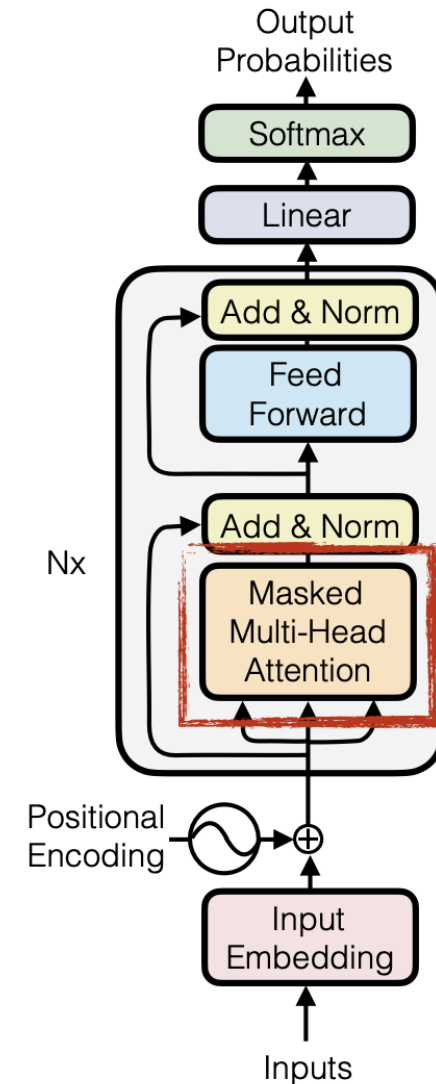
I **run** a **mile** in 10 minutes

The **robber** made **a run** for it

The **stocking** had **a run**

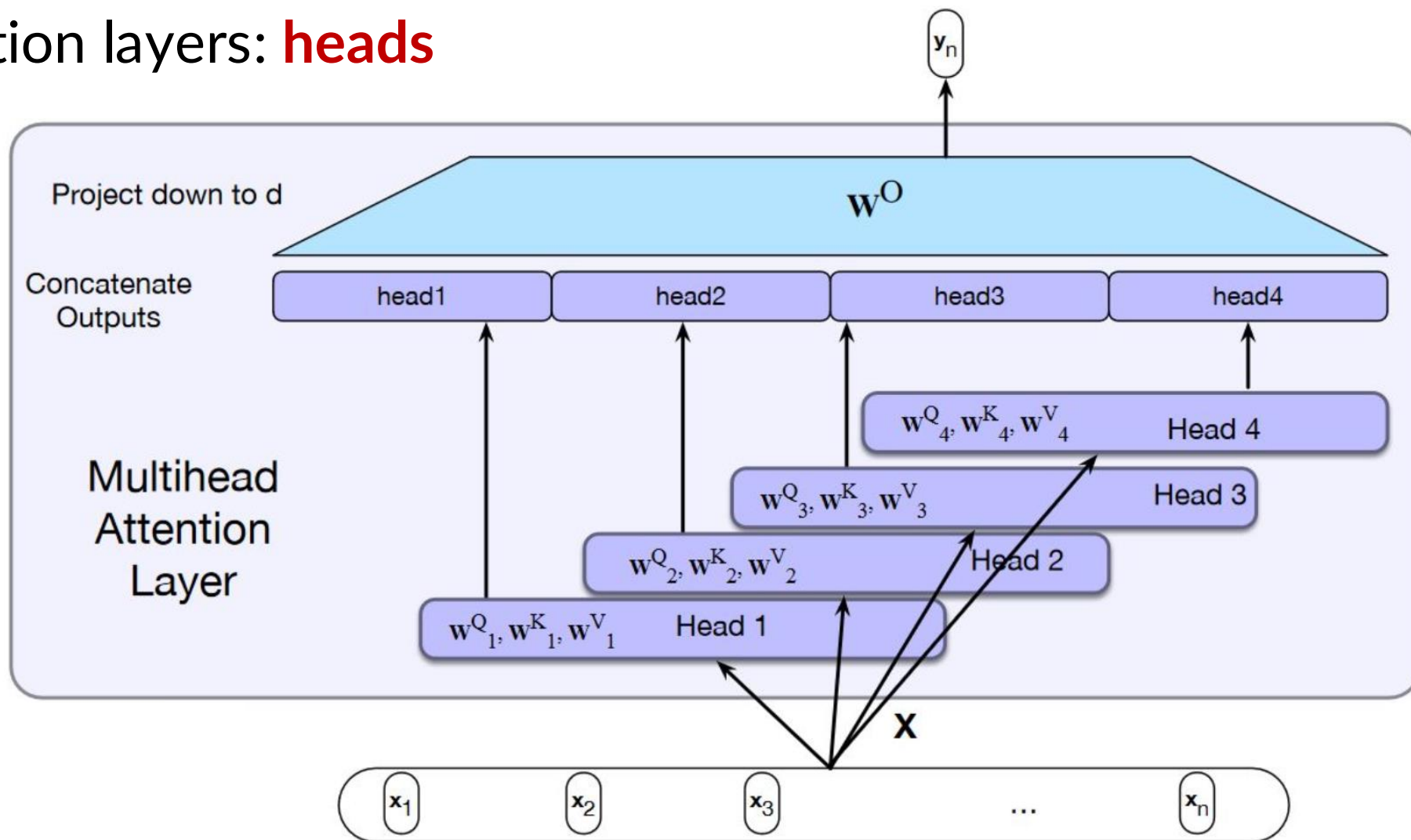
syntax
(nearby context)

semantics
(farther context)



Architecture of Multi-heads

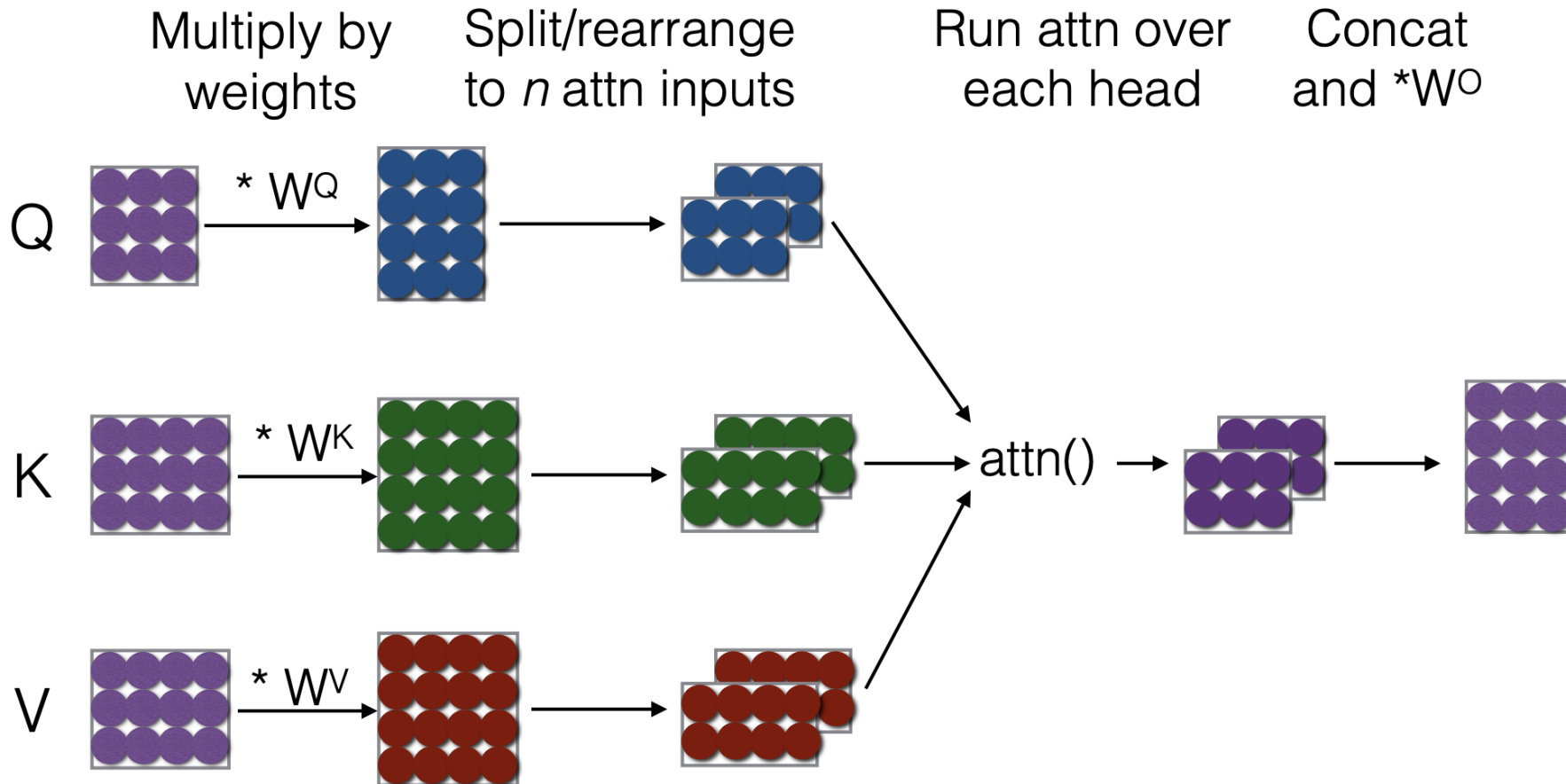
- Set of self-attention layers: **heads**



Calculate Multi-heads

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



Code Example

```
def forward(self, query, key, value, mask=None):
    nbatches = query.size(0)

    # 1) Do all the linear projections
    query = self.W_q(query)
    key = self.W_k(key)
    value = self.W_v(value)

    # 2) Reshape to get h heads
    query = query.view(nbatches, -1, self.heads, self.d_k).transpose(1, 2)
    key = key.view(nbatches, -1, self.heads, self.d_k).transpose(1, 2)
    value = value.view(nbatches, -1, self.heads, self.d_k).transpose(1, 2)

    # 3) Apply attention on all the projected vectors in batch.
    x, self.attn = attention(query, key, value)

    # 4) "Concat" using a view and apply a final linear.
    x = (
        x.transpose(1, 2)
        .contiguous()
        .view(nbatches, -1, self.h * self.d_k)
    )
    return self.W_o(x)
```

What Happens with Multi-heads?

Example from Vaswani et al.

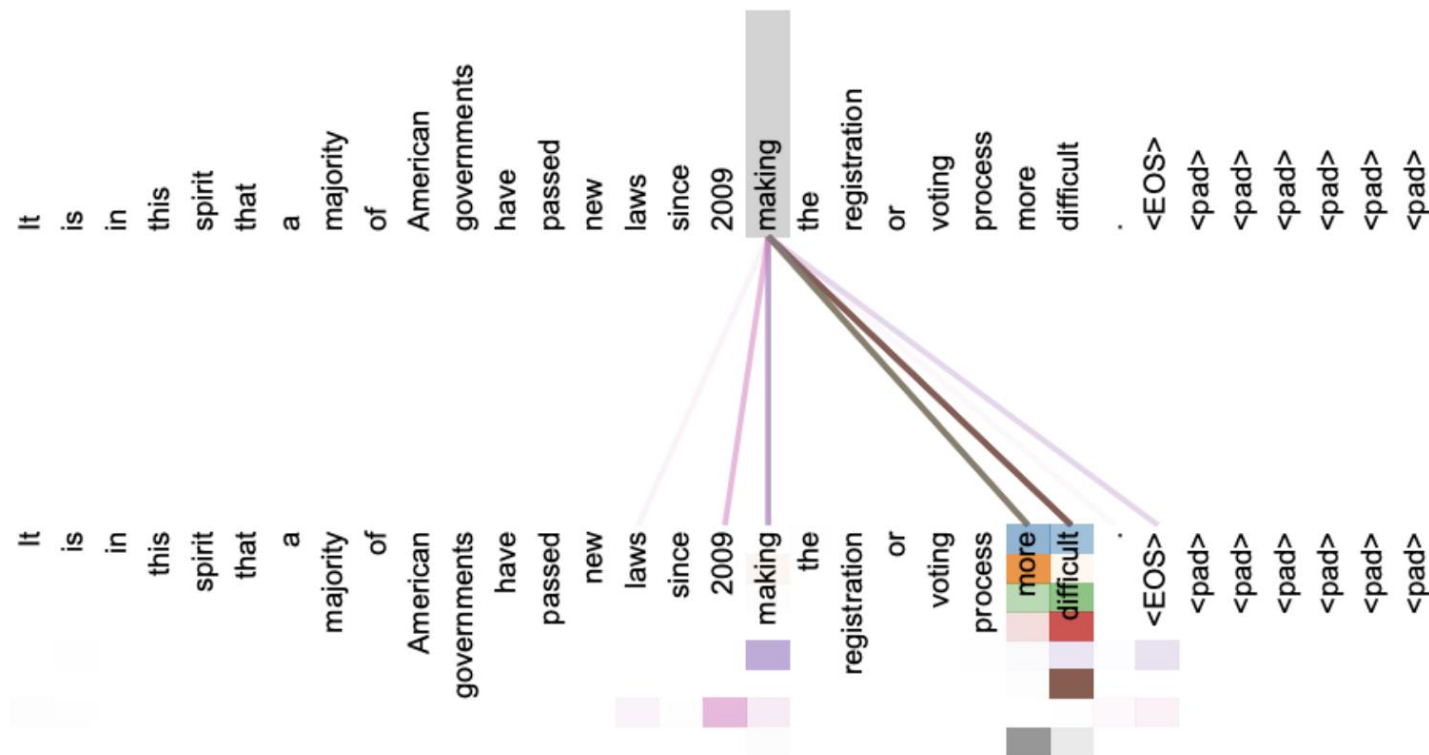
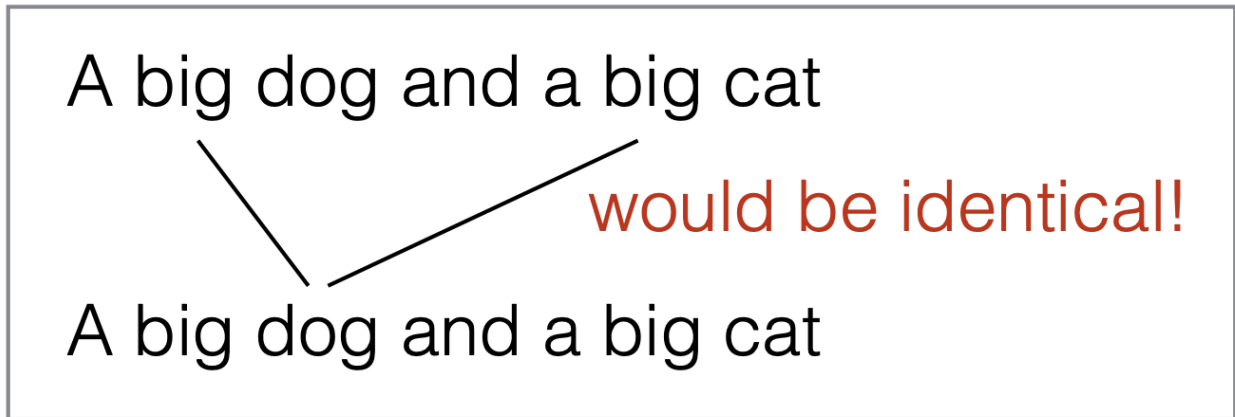


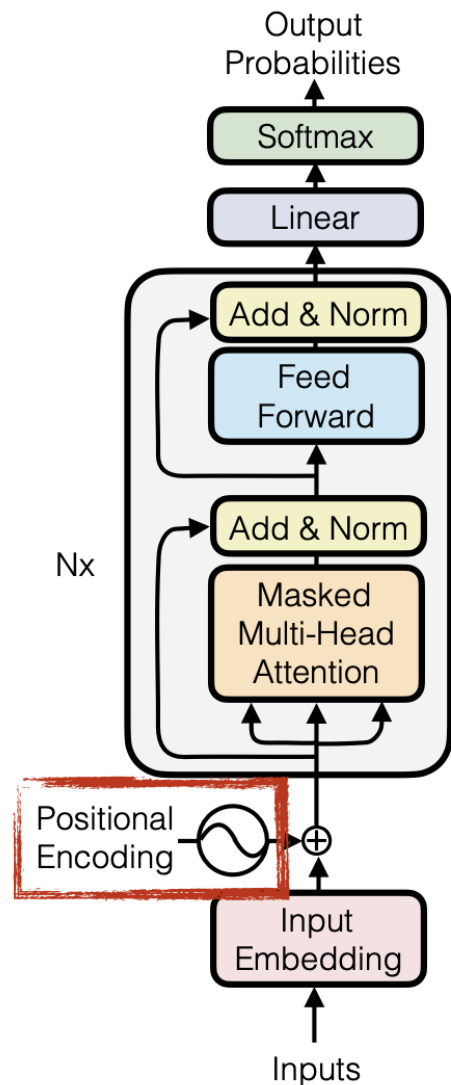
Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

Positional Encoding

- The Transformer model is purely attentional
- If embeddings were used, there would be no way to distinguish between identical words

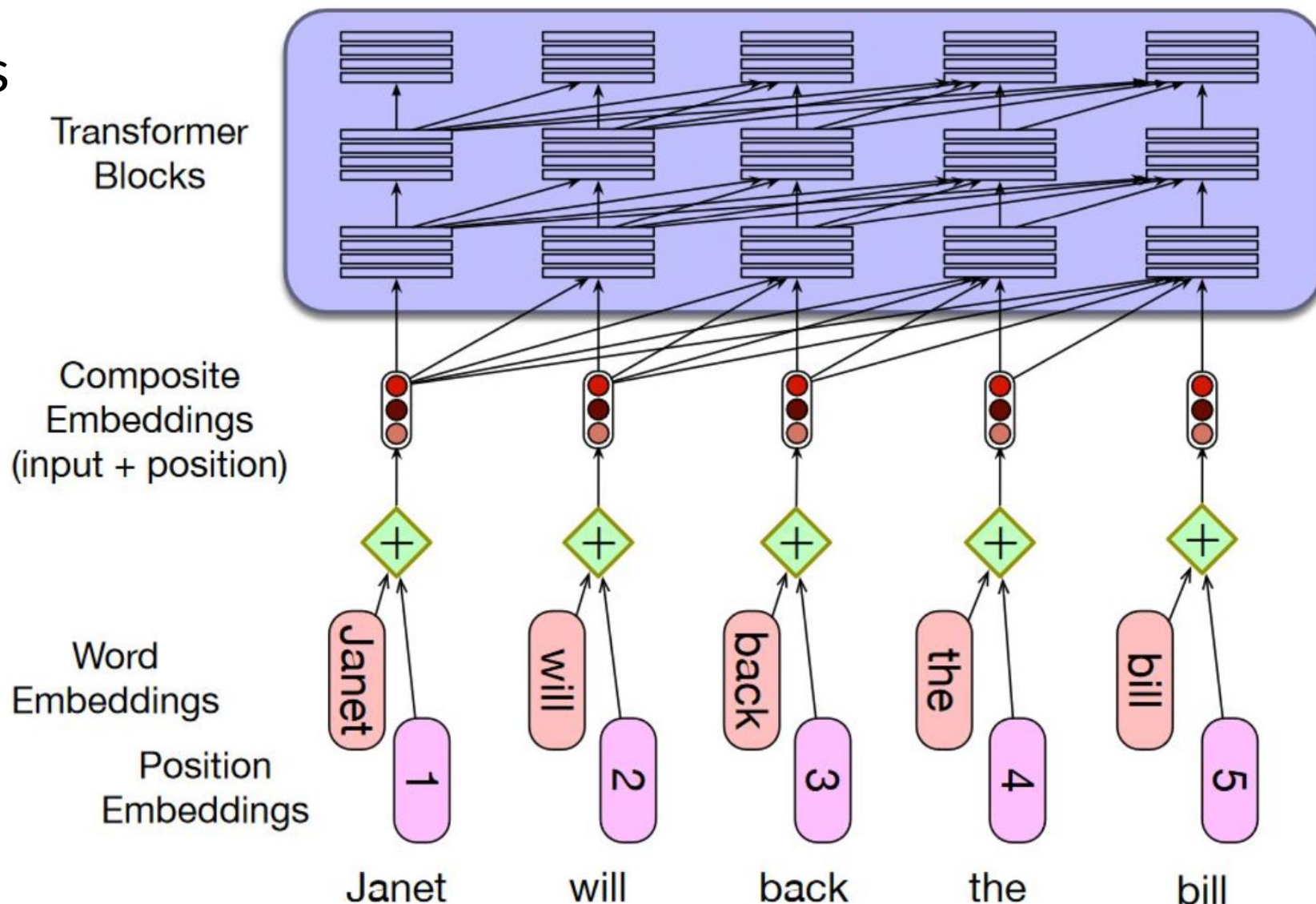


- Positional encodings add an embedding based on the word position
- $W_{\text{big}} + W_{\text{pos}2}$ $W_{\text{big}} + W_{\text{pos}6}$



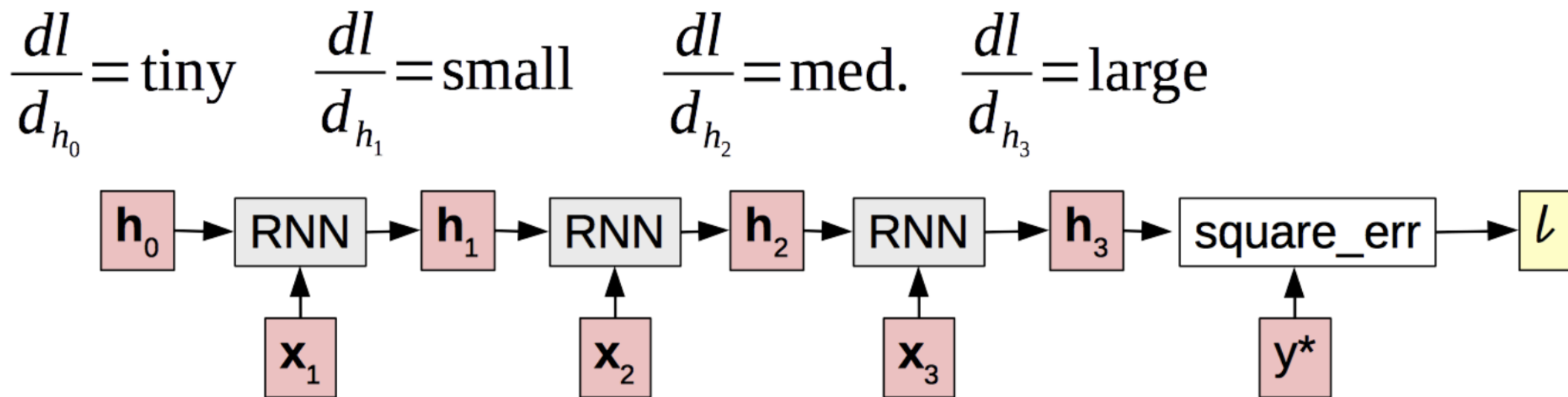
Architecture of the CBOW model

- Positional embeddings are learned along with other parameters during training
- Simple way: **absolute position**



Layer Normalization and Residual Connections

- In RNNs, we asked about how back-propagation through a network causes gradient can vanish or explode



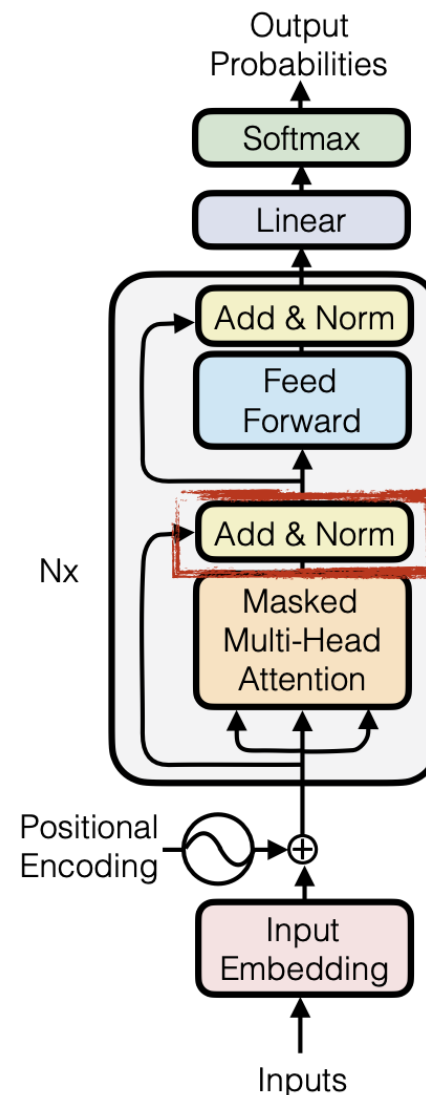
- The same issue occurs in multi-layer transformers

Layer Normalization

- Normalizes the outputs to be within a consistent range, preventing too much variance in scale of outputs

$$\text{LayerNorm}(\mathbf{x}; \mathbf{g}, \mathbf{b}) = \underbrace{\frac{\mathbf{g}}{\sigma(\mathbf{x})}}_{\text{vector stddev}} \odot (\mathbf{x} - \underbrace{\mu(\mathbf{x})}_{\text{vector mean}}) + \underbrace{\mathbf{b}}_{\text{bias}}$$

$$\mu(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n x_i \quad \sigma(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

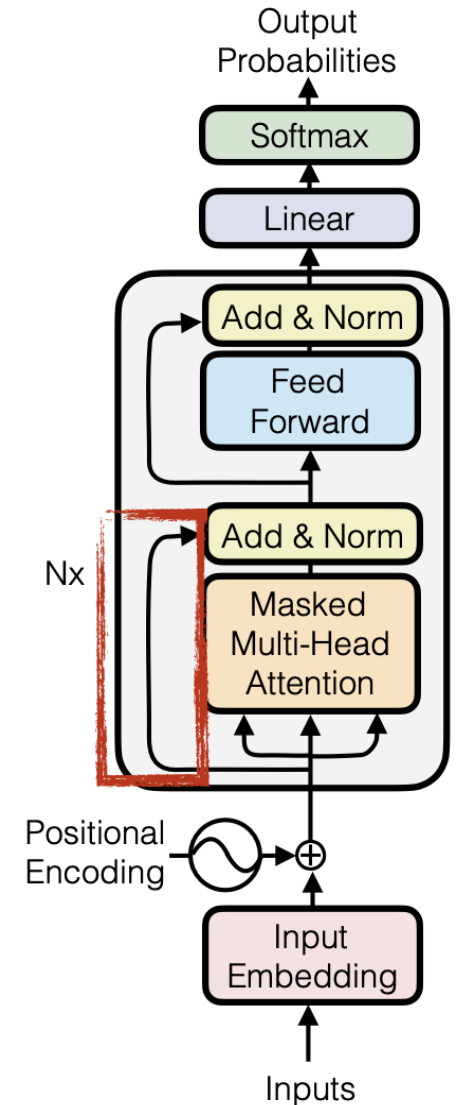


Residual Connections

- Add an additive connection between input and output

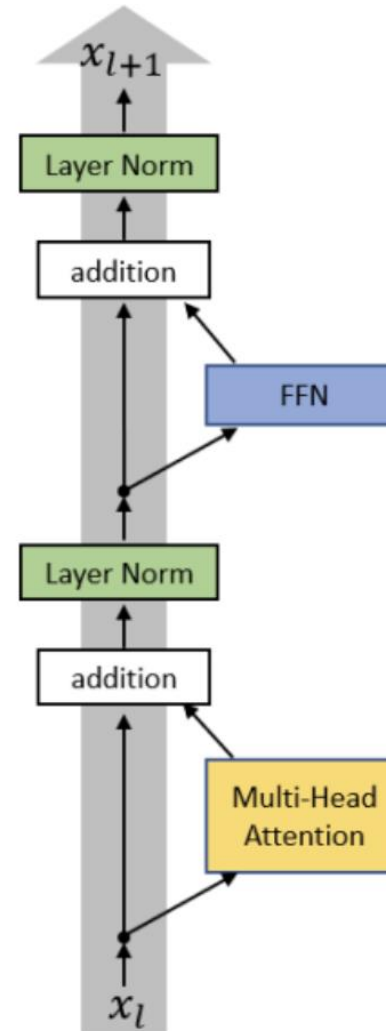
$$\text{Residual}(\mathbf{x}, f) = f(\mathbf{x}) + \mathbf{x}$$

- Prevent vanishing gradients and allow f to learn the difference from input
- *What are the implications for self-attention w/ and w/o residual connections?*

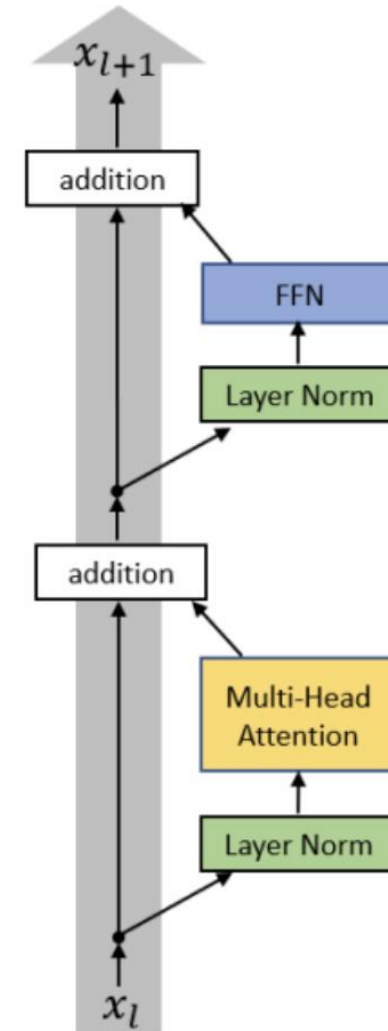


Post- vs. Pre-Layer Norm

- Pre-layer Norm is **better** for gradient propagation



post-LayerNorm

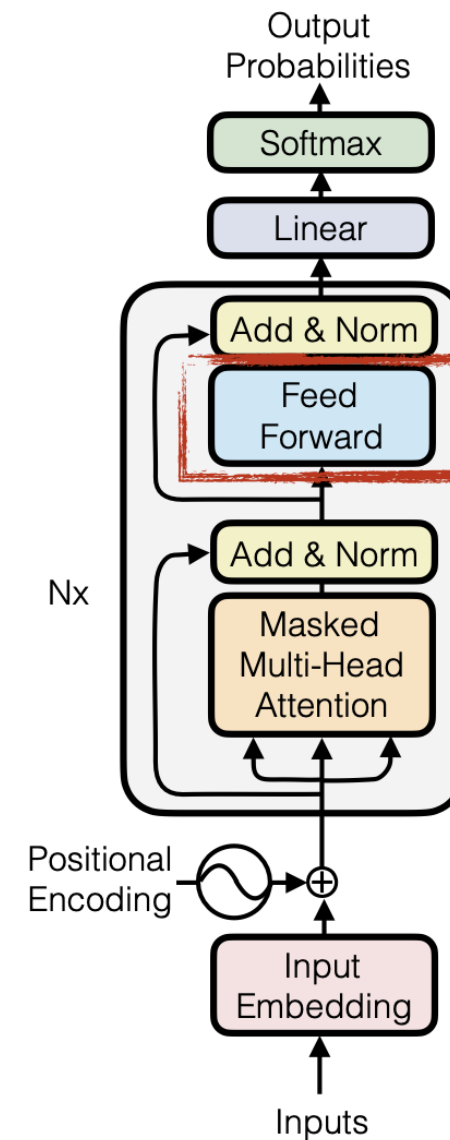
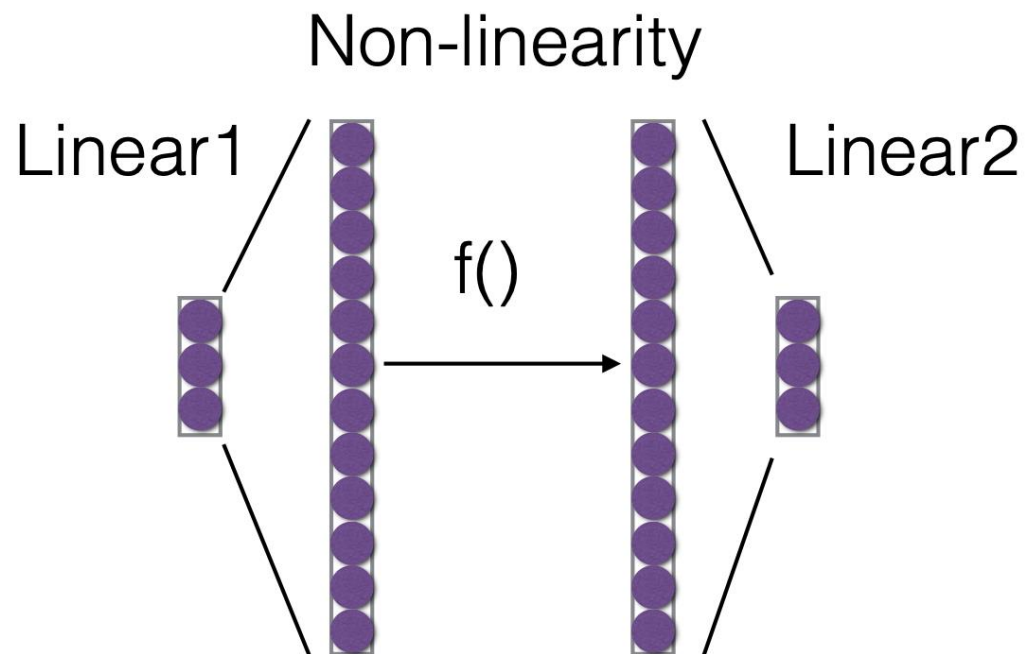


pre-LayerNorm

Feed Forward Layers

- Objective: extract combination features from attended outputs

$$\text{FFN}(x; W_1, \mathbf{b}_1, W_2, \mathbf{b}_2) = f(\mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2$$



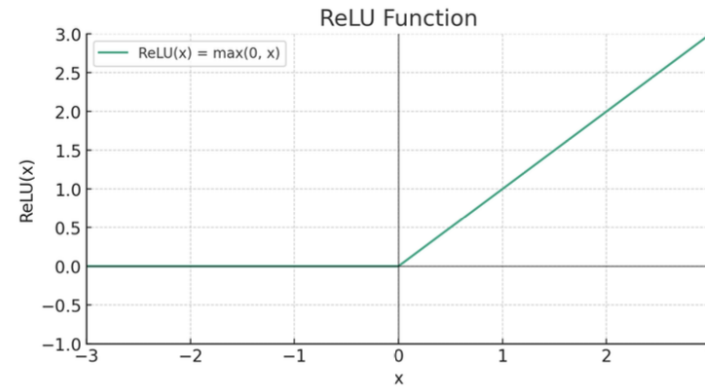
Self-Attention vs. Feed forward

- Self-Attention: looking around to gather relevant information
- Feed-forward: thinking through what to do with that information at each token
- Self-Attention is mostly linear, FFNs introduce non-linear transformation to the representations
- Unlike self-attention which mixes information across position, FFNs apply independently and identically to each position → encode position-specific transformations

Activation Functions in FFN

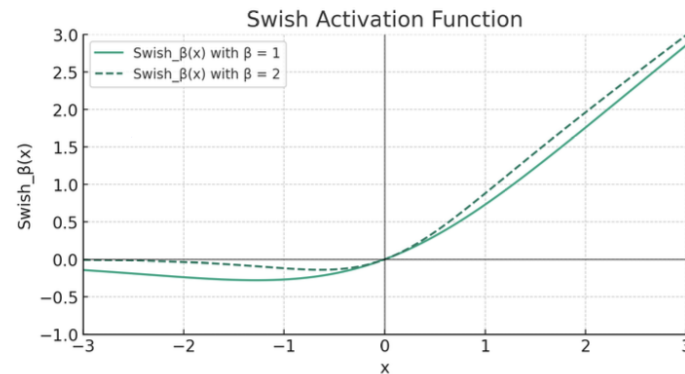
- Vaswani et al.: ReLU

$$\text{ReLU}(\mathbf{x}) = \max(0, \mathbf{x})$$



- LLaMa: Swish/SiLU (Hendricks and Gimpel 2016)

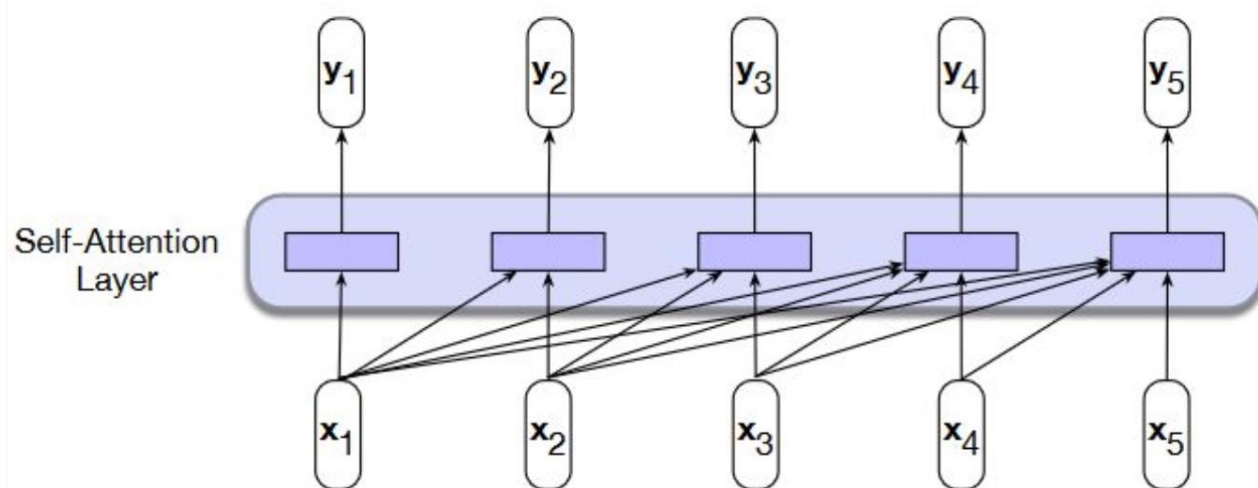
$$\text{Swish}(\mathbf{x}; \beta) = \mathbf{x} \odot \sigma(\beta \mathbf{x})$$



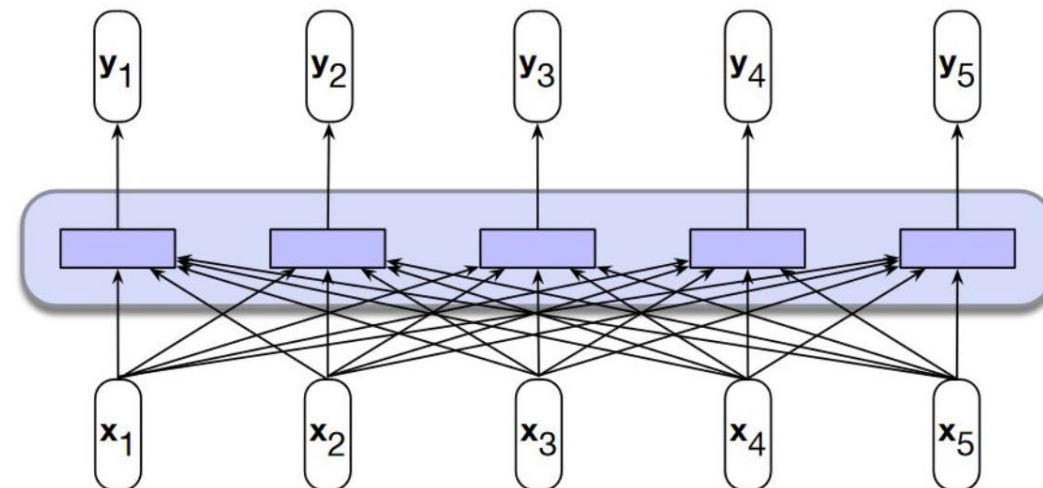
Section 2 – Pre-trained Language Models

ONE LOVE. ONE FUTURE.

BERT – Bidirectional Encoder Representations from Transformers



backward looking, transformer model

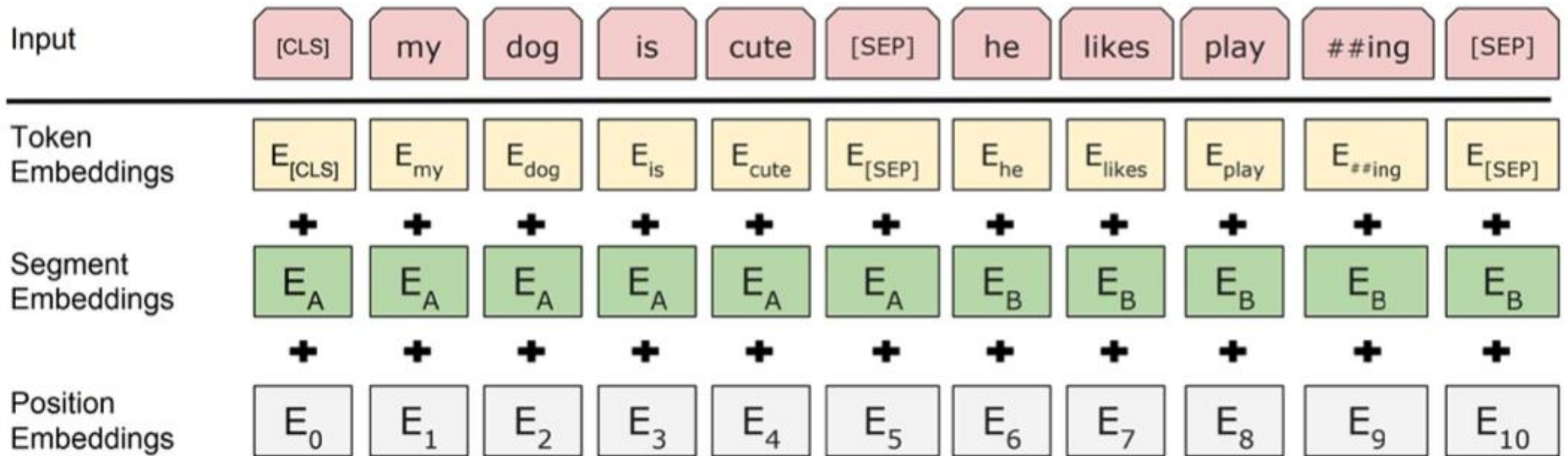


bidirectional transformer model

Computing contextualized representations of the tokens in two directions
➔ Enrich the representations

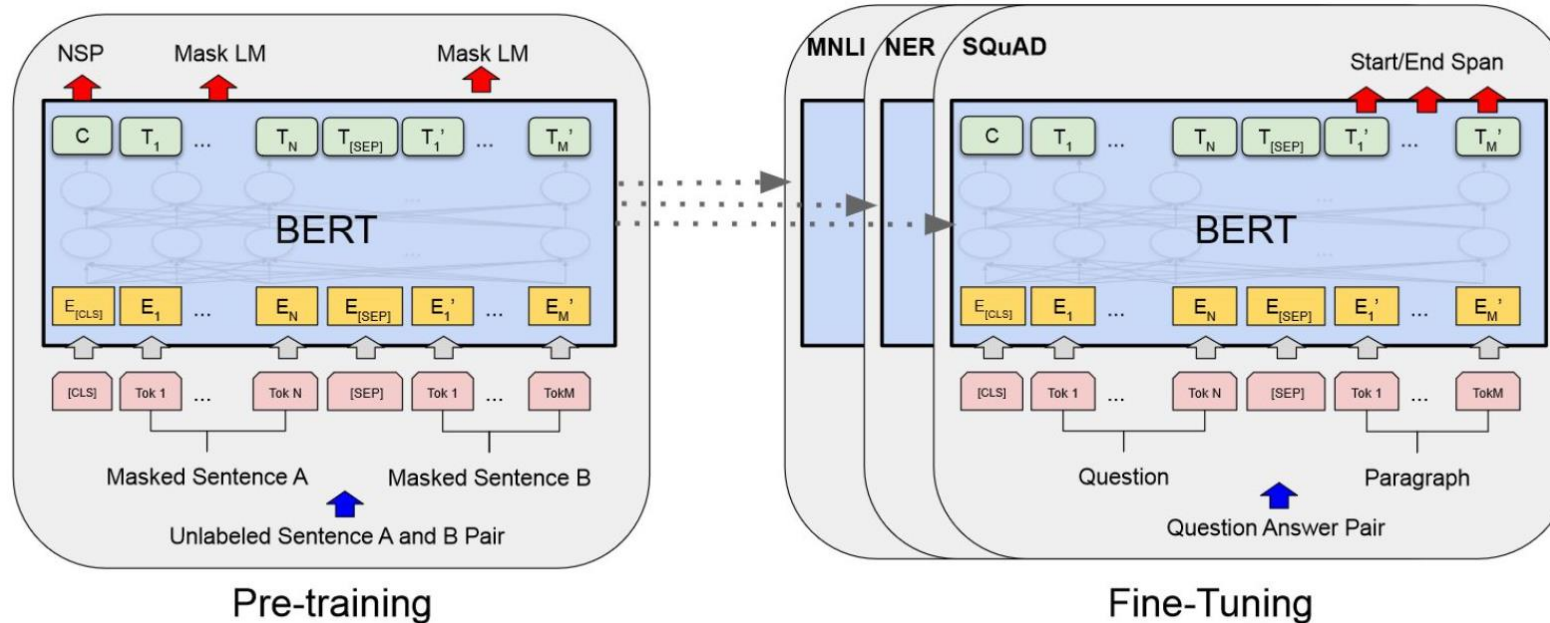
BERT - Overview

- BERT makes use of Transformer
- BERT's goal is to generate a language model, only the encoder mechanism is necessary
- Use 30,000 Word-Piece vocabulary on input
- Each token is sum of **three embeddings**



[CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token

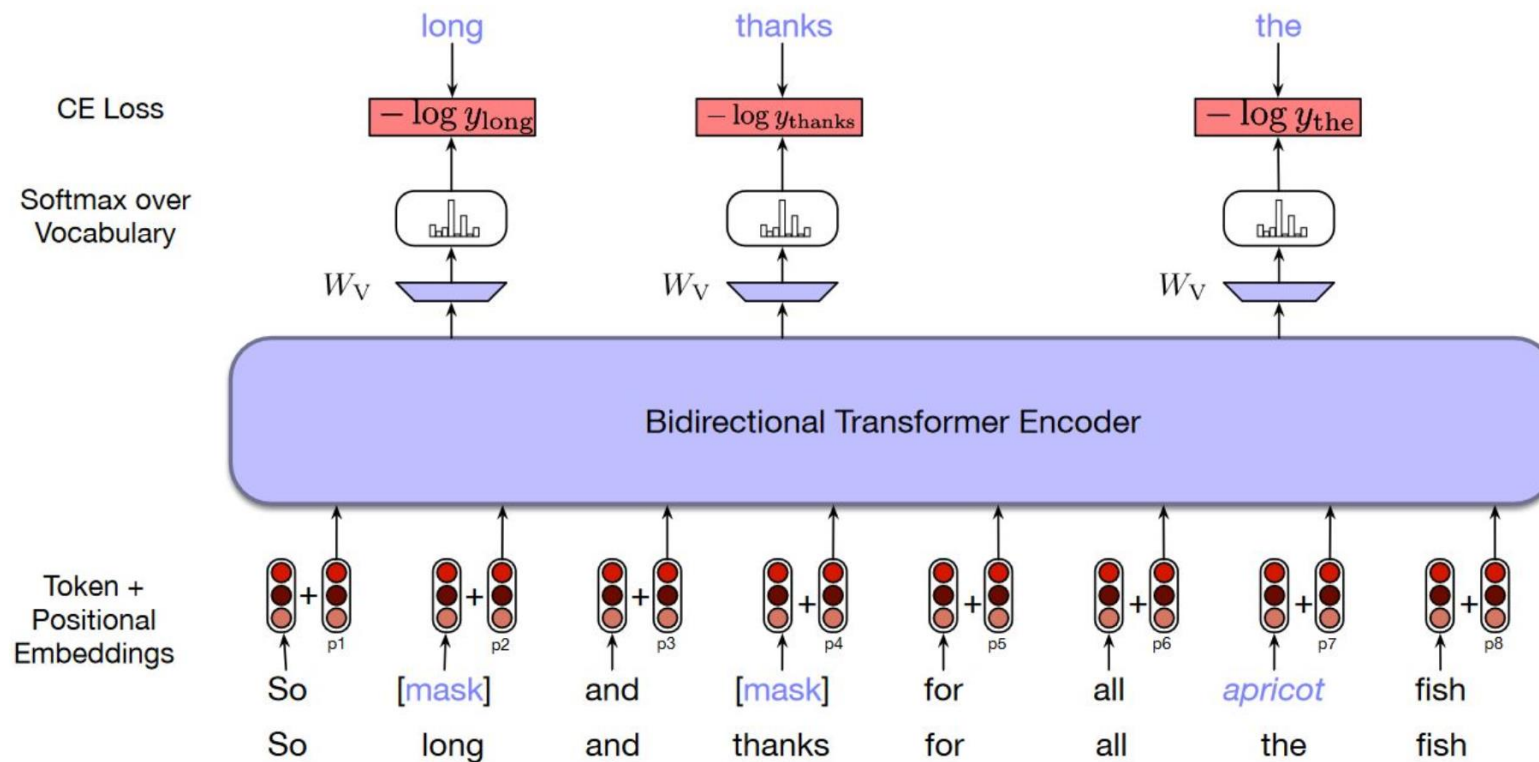
Pre-Training vs Fine-tuning



Pre-training: two tasks are considered

1. **Masked LM:** mask some percentage of the input tokens at random, and then predict those masked tokens
Mask 15% of all WordPiece tokens in each sequence at random
2. **Next Sentence Prediction:** understanding the relationship between two sentence (50% of positive pairs)
Used BooksCorpus (800M words) and Wikipedia (2,500M words)

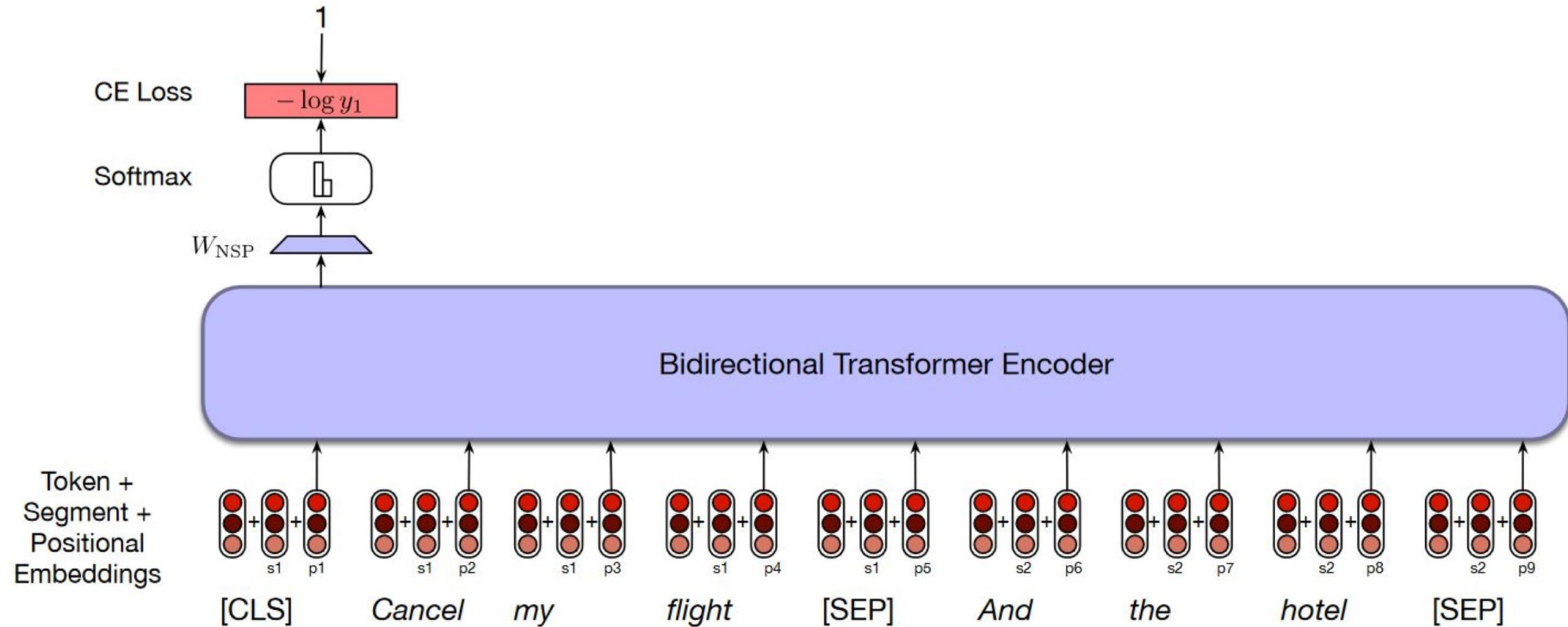
Masked Language Model



In BERT, 15% of the input tokens in a training sequence are sampled for learning

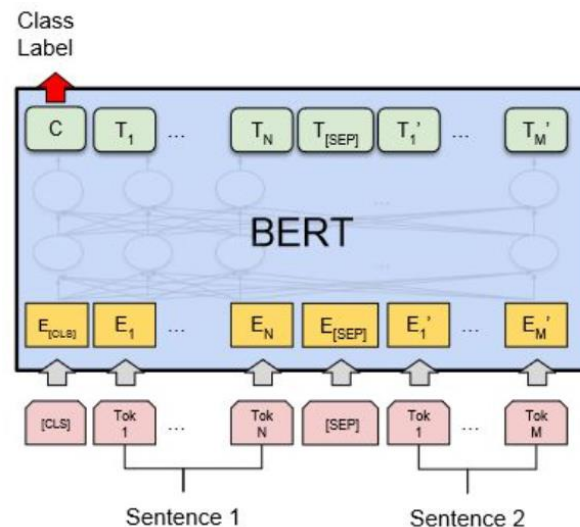
- Of these, 80% are replaced with [MASK], 10% are replaced with randomly selected tokens, and the remaining 10% are left unchanged

Next Sentence Prediction

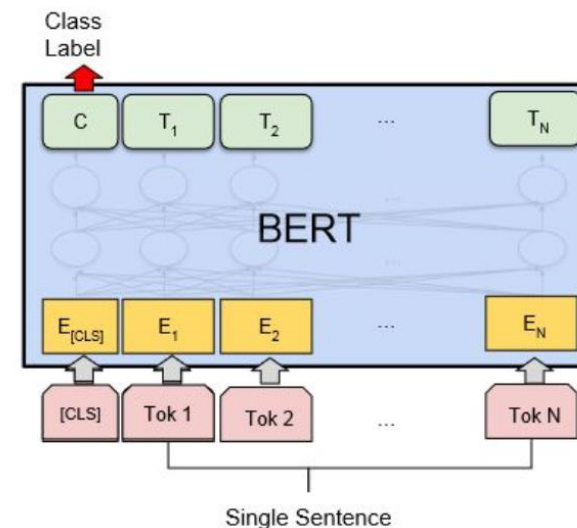


Training: The model is presented with pairs of sentences and is asked to predict whether each pair consists of an actual pair of adjacent sentences from the training corpus or a pair of unrelated sentences

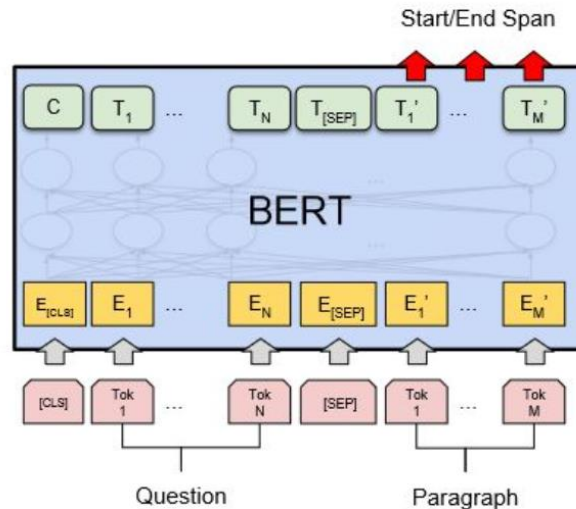
Fine-Tuning



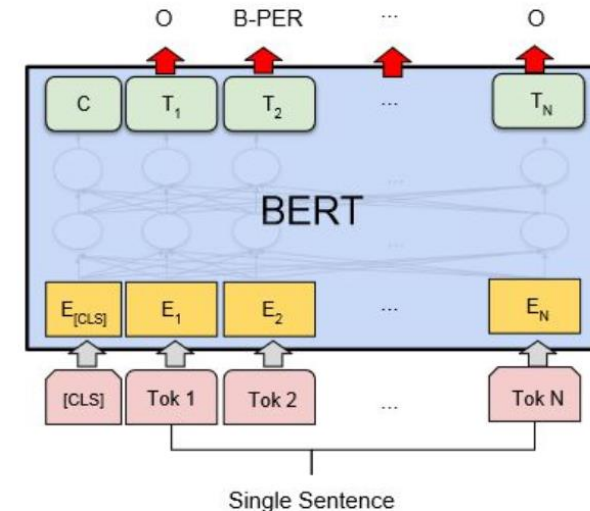
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1

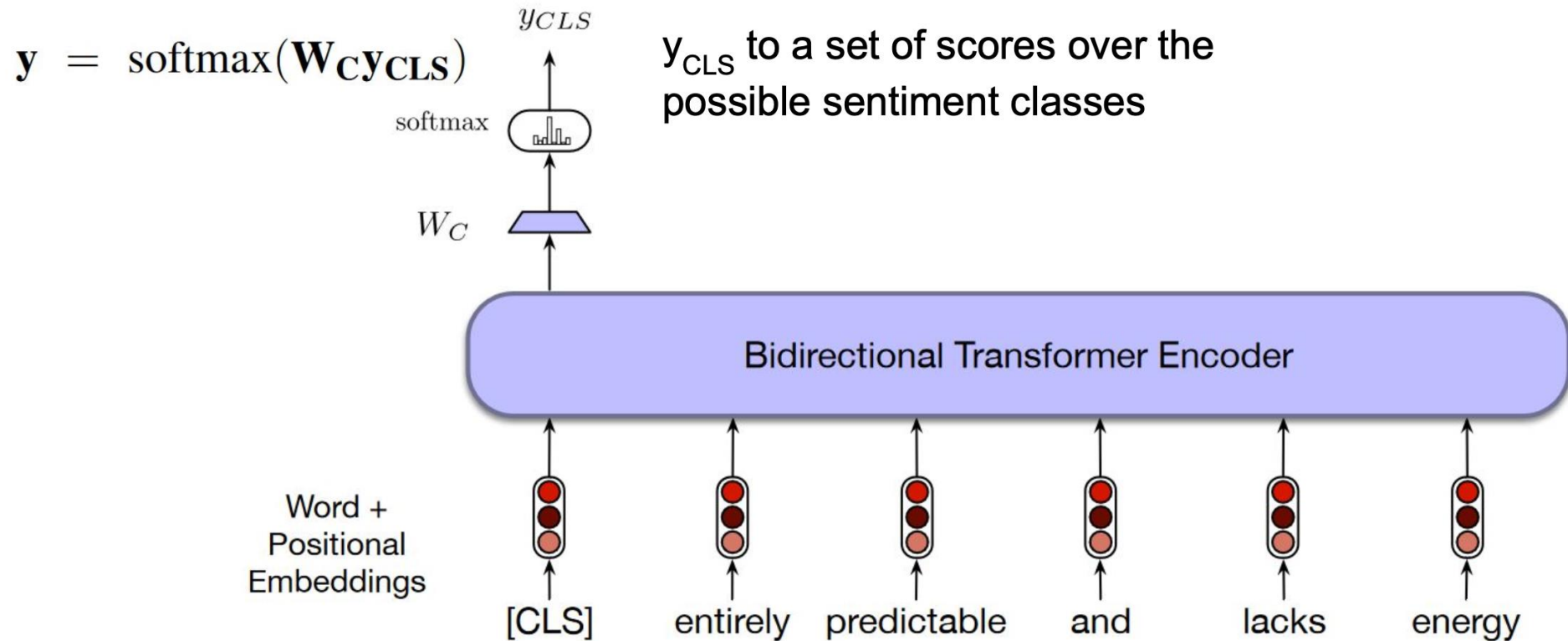


(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Tasks:

- Question answering
- Abstract summarization
- Sentence prediction
- Conversational response generation
- Sentiment classification

Sentiment Classification



Learning a set of weights, $\mathbf{W}_C$, to map the output vector for the [CLS] token, $\mathbf{y}_{CLS}$

The output vector in the final layer of the model for the [CLS] input represents the entire input sequence and serves as the input to a classifier head

RoBERTa: Robustly Optimized BERT Approach

- Facebook AI research team
- They used 160 GB of text instead of the 16 GB dataset originally used to train BERT
- Increased the number of iterations from 100K to 300K and then further to 500K
- Dynamically changing the masking pattern applied to the training data
- Removing the next sequence prediction objective from the training procedure

ALBERT, XLNET, CoBERT

Generative Pre-trained Transformer (GPT)

GPT (Generative Pre-trained Transformer) is a series of language generation models developed by OpenAI. These models are based on the Transformer architecture (2018)

GPT-2 (Generative Pre-trained Transformer 2) was the second model in the GPT series, released in 2019. It was trained on a large corpus of internet text and was designed for language generation tasks such as question answering, and text summarization

GPT-3 (Generative Pre-trained Transformer 3) Released in 2020, with over 175 billion parameters, and was trained on a much larger and diverse dataset, including web pages, books, and scientific articles

GPT-4 (Generative Pre-trained Transformer 4) Released in 2023, a multimodal model which can accept image and text inputs and produce text outputs

Zero-shot, One-shot and Few-shot, Contrasted with Traditional Fine-tuning

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Language Models are Few-Shot Learners



Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



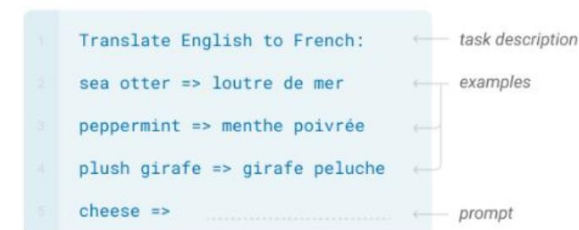
One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

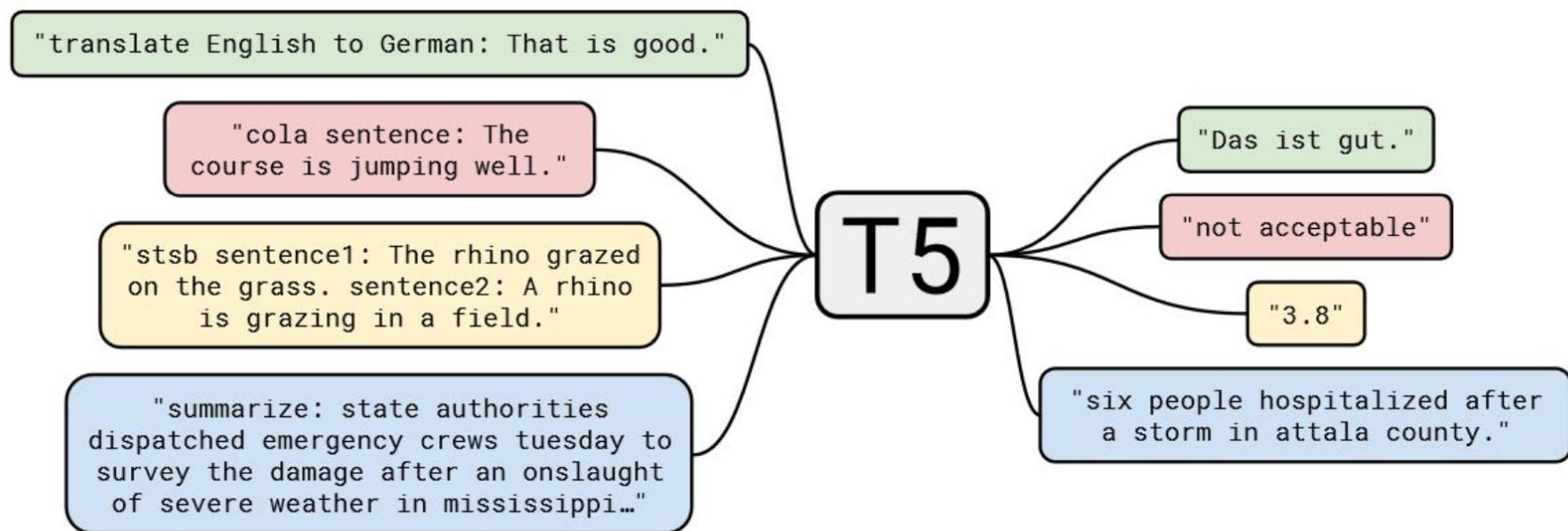


Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Text-to-Text Transfer Transformer(T5)



A diagram of the text-to-text framework ([T5](#))

Every task, including translation, question answering, and classification, is cast as feeding T5 model text as input and training it to generate some target text

T5 vs. BERT/GPT

Model	Direction	Input/Output	Pretraining	Use Cases
BERT	Encoder-only	Input: text → [CLS]	Masked LM	Classification, QA
GPT	Decoder-only	Input: text → output	Causal LM	Text generation
T5	Encoder-Decoder	Input: text → Output: text	Span corruption	Any text-to-text task

Training objective of T5: Span Corruption

- The model is learned how to **reconstructing** missing spans (chunks of text) from the input sentences
- BERT masks individual tokens by [MASK], T5 randomly selects a span of text and replace it by a unique place holder token <extra_id_0>, <extra_id_1>, etc.

Original text

The quick brown fox jumps over the lazy dog.

Masked spans

The <extra_id_0> fox jumps over <extra_id_1> dog.

Target output

<extra_id_0> quick brown <extra_id_1> the lazy <extra_id_2>



Transformer and Pre-trained Language Models

(end of lecture)

ONE LOVE. ONE FUTURE.